NAME :G.AKSHAYA

REG NO: 192521156

COURSE CODE : CSA0619

COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHMS

CO5 - ASSESSMENT TOOL – 02

Q2. Subset Sum Problem

Aim:

To solve the Subset Sum Problem using the Backtracking technique and determine whether a subset of the given set of integers exists whose sum is equal to the specified target value.

Problem Statement:

Given a set of integers and a target sum, determine whether there exists a subset whose elements add up exactly to the target value. The problem is solved using Backtracking, where each element has two choices: either it can be included in the subset or excluded from the subset.

The algorithm explores different possible combinations and uses pruning to stop branches that cannot produce the required target.

Objective:

- To determine whether a valid subset exists for a given target sum.
- To apply the Backtracking technique to the Subset Sum problem.
- To explore the include and exclude choices for each element.
- To identify and display a subset whose sum equals the target.
- To reduce unnecessary computation using pruning.
- To understand recursive problem solving.
- To analyze the time and space complexity.

Input:

- A set of n integers.
- A target sum T.

Output:

- `TRUE` if a subset with sum equal to `T` exists.
- `FALSE` if no such subset exists.
- The selected subset can also be displayed.

Example:

Consider:

A = {10, 7, 5, 18, 12, 20, 15}

Target = 35

A valid subset is:

{10, 5, 20}

Verification:

10 + 5 + 20 = 35

Therefore:

Subset Sum = TRUE

Backtracking Approach :

Backtracking solves the problem by considering every element one at a time.

For each element, there are two possibilities:

1. Include

The current element is included in the subset.

Current Sum = Current Sum + Element

2. Exclude

The current element is not included.

Current Sum remains unchanged

The algorithm recursively explores both choices until it finds a valid subset.

Working Example :

For:

A = {10, 7, 5}

Target = 15

The algorithm starts with an empty subset:

{}

It considers `10`.

Include 10 → {10}

Exclude 10 → {}

From `{10}`, it considers `7`:

{10,7}

Then it considers `5`:

10 + 5 = 15

The target is reached.

Therefore:

Subset = {10,5}

Sum = 15

State Representation :

The recursive function maintains:

index

current_sum

subset

Where:

- index → Current position in the array.
- current_sum → Sum of selected elements.
- subset → Elements selected so far.

A solution is obtained when:

current_sum == target

Pruning :

Pruning is used to stop unnecessary branches.

For non-negative input values, if:

current_sum > target

the current branch can be stopped because adding more positive values cannot bring the sum back to the target.

For example:

Target = 35
Current Sum = 40

Since:

40 > 35

the branch is discarded.

This improves practical execution time by avoiding unnecessary searches.

Pseudocode :

```
SUBSET-SUM(A, n, T)

  BACKTRACK(index, currentSum, subset)

    IF currentSum = T
      PRINT subset
      RETURN TRUE

    IF index = n OR currentSum > T
      RETURN FALSE

    ADD A[index] TO subset
```

IF BACKTRACK(index + 1,
currentSum + A[index],
subset)
RETURN TRUE

REMOVE A[index] FROM subset

IF BACKTRACK(index + 1,
currentSum,
subset)
RETURN TRUE

RETURN FALSE

RETURN BACKTRACK(0, 0, empty set)

SOURCE CODE:

main.py

Visualize

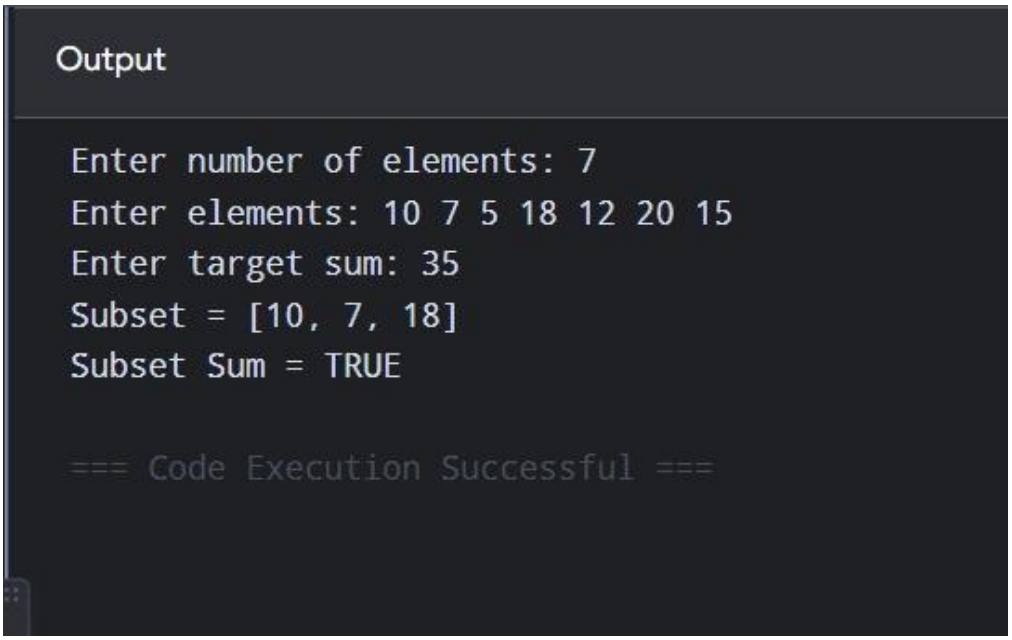
```
def subset_sum(arr,target):
    def backtrack(index,current_sum,subset):
        if current_sum==target:
        if index==len(arr) or current_sum>target:
            return False
        subset.append(arr[index])
        if backtrack(index+1,current_sum+arr[index],subset):
            return True
        subset.pop()
        if backtrack(index+1,current_sum,subset):
            return True
        return False

    return backtrack(0,0,[])

n=int(input("Enter number of elements: "))
arr=list(map(int,input("Enter elements: ").split()))
target=int(input("Enter target sum: "))

if subset_sum(arr,target):
    print("Subset Sum = TRUE")
else:
    print("Subset Sum = FALSE")
```

OUTPUT:



Output Verification:

10 + 5 + 20 = 35

Therefore, the selected subset satisfies the target sum.

Complexity Analysis:

For n elements, every element can either be included or excluded.

Therefore, the maximum number of possible subsets is:

2^n

Time Complexity:

Worst Case: $O(2^n)$

The algorithm may need to examine almost every possible subset.

Space Complexity:

O(n)

This is required for the recursion stack and the currently selected subset.

Advantages :

- Simple and easy to understand.
- Finds an exact solution.
- Can display the actual subset.
- Include/exclude decisions provide a systematic search.
- Pruning can reduce unnecessary computation.
- Suitable for small input sizes.

Limitations :

- Worst-case time complexity is exponential.
- Performance decreases rapidly as the number of elements increases.
- Large datasets may require considerable computation.
- Basic pruning is most suitable for non-negative integers.
- Not ideal for very large input sets.

Applications :

- Resource selection problems.
- Budget allocation.
- Production planning.
- Package and cargo selection.
- Financial decision-making.
- Combinatorial optimization.
- Finding combinations that satisfy a required total.

Result:

The Subset Sum Problem was successfully solved using the Backtracking technique. The algorithm considers both inclusion and exclusion of each element and uses pruning to eliminate unnecessary branches. For the given input `{10, 7, 5, 18, 12, 20, 15}` with target `35`, the subset `{10, 5, 20}` is found, whose sum is exactly 35. Therefore, the result is TRUE.